Übung 5

Methoden

Programmierung 1 · WS 2026/27
Bachelor Cyber Security & Privacy · André Kless, M.Sc.

Ziel: Algorithmen in wiederverwendbare Methoden zerlegen, Parameter und Rückgaben bewusst wählen sowie Wertübergabe und Rekursion nachvollziehen.

Arbeitsweise: Verwenden Sie Java 25 oder höher. Legen Sie im Projekt `Prog1` je Aufgabe ein Compact File an, z. B. `K05Aufgabe01.java`. Eigene Methoden stehen neben `void main()`, nicht darin. Nutzen Sie die Folien aus Kapitel 5 zum Nachschlagen.

Reihenfolge: Basic (1–4) festigt die Grundlagen, Combined (5–8) verbindet sie, Challenge (9–10) vertieft Rekursion und die Zerlegung eines größeren Problems.

Tests: Bestimmen Sie erwartete Ergebnisse zuerst selbst. Sie dürfen die jeweils angegebene Eingabeform und den Wertebereich voraussetzen. Prüfen Sie absichtlich ungültige Codezeilen getrennt.

01 Methoden als Blackbox entwerfen | Basic

a) Entwerfen Sie für jeden Auftrag einen Methodenkopf und einen passenden Aufruf: ein festes Menü ausgeben; das Volumen eines Quaders aus drei `double`-Seitenlängen berechnen; einen übergebenen Text mehrfach ausgeben; eine ganze Zahl von der Tastatur einlesen und zurückgeben.

b) Begründen Sie jeweils Parameter und Rückgabetyp: Was braucht die Methode, um ihren Auftrag zu erledigen? Welche Informationen muss der Aufrufer liefern? Wo entsteht eine Bildschirmausgabe statt eines Rückgabewerts?

c) Implementieren Sie die Volumenberechnung und die Menüausgabe. Rufen Sie beide aus `main` auf. Prüfen Sie die Volumenberechnung mit den Seitenlängen 2, 3 und 4 sowie 1,5, 2 und 3. Die Seitenlängen sind positiv.

02 Rückgabe und Bildschirmausgabe unterscheiden | Basic

```
int verdopple(int zahl) {
    return 2 * zahl;
}

void zeigeDoppeltes(int zahl) {
    IO.println(2 * zahl);
}
```

a) Welche Ausgaben erzeugt der folgende Ausschnitt in `main`? Was geschieht mit dem Ergebnis des ersten Methodenaufrufs?

```
int a = 4;
verdopple(a);
IO.println(a);
IO.println(verdopple(a + 1));
```

b) Warum ist `int b = zeigeDoppeltes(3);` ungültig? Ändern Sie den Aufruf so, dass das doppelte Ergebnis in `b` gespeichert wird.

c) Ergänzen Sie `zeigeDoppeltes` so, dass die Methode bei einer negativen Zahl durch `return;` ohne Ausgabe endet. Geben Sie in `main` vor und nach einem solchen Aufruf je einen Text aus. Endet auch `main` durch dieses `return;`?

Basic: Wiederverwendung und Überladung

03 Code auslagern und dokumentieren | Basic

In `main` werden zwei Rechteckflächen berechnet. Alle Seitenlängen sind ganze Zahlen von 1 bis 1000 und werden in Zentimetern angegeben.

```
int breite = 3;
int hoehe = 4;
int flaeche = breite * hoehe;
IO.println(flaeche);
breite = 5;
hoehe = 2;
flaeche = breite * hoehe;
IO.println(flaeche);
```

a) Lagern Sie die erste Flächenberechnung mit **Refactor → Extract → Method** in eine Methode `berechneFlaeche` aus. Die Methode erhält die beiden Seitenlängen und gibt die Fläche zurück. Ersetzen Sie auch die zweite Berechnung durch einen Aufruf. Die Ausgaben bleiben in `main`.

b) Benennen Sie die Methode über **Refactor → Rename** in `rechteckFlaeche` um. Prüfen Sie, ob die IDE sowohl die Deklaration als auch beide Aufrufe angepasst hat. Erklären Sie den Bezug zum DRY-Prinzip.

c) Schreiben Sie einen Doc-Comment mit `/** ... */` unmittelbar vor die Methode. Beschreiben Sie ihren Zweck, beide Parameter mit `@param` und das Ergebnis mit `@return`. Nennen Sie Einheiten und erlaubte Werte.

d) Rufen Sie die Methode zusätzlich mit festen Zahlen, mit zwei eingelesenen Werten und mit einem Ausdruck als Argument auf. Verwenden Sie in `main` andere Variablennamen als in der Parameterliste. Warum funktioniert das trotzdem?

04 Methoden überladen | Basic

Schreiben Sie drei Methoden mit demselben Namen:

```
int minimum(int a, int b)
int minimum(int a, int b, int c)
double minimum(double a, double b)
```

a) Implementieren Sie alle drei Varianten. Die Variante mit drei Parametern soll die Variante mit zwei Parametern wiederverwenden. Verwenden Sie keine fertige Minimum-Methode.

b) Welche Variante wird bei den folgenden Aufrufen gewählt? Geben Sie jeweils Ergebnis und Rückgabetyp an: `minimum(3, 7)`, `minimum(3, 7, 5)`, `minimum(3.5, 7.2)` und `minimum(3, 7.2)`. Prüfen Sie zusätzlich negative und gleiche Werte.

c) Kann eine weitere Überladung dieselben Parametertypen, aber einen anderen Rückgabetyp haben? Genügen andere Parameternamen? Begründen Sie beides anhand der Methodensignatur.

Combined: Wertübergabe und Ablauf verstehen

05 Wertübergabe im Debugger verfolgen | Combined

```
void main() {
    int zahl = 5;
    String text = "Hallo";
    aendere(zahl, text);
    IO.println(zahl);
    IO.println(text);
}

void aendere(int wert, String nachricht) {
    wert = 99;
    nachricht = nachricht + "!";
    IO.println(wert);
    IO.println(nachricht);
}
```

a) Notieren Sie die vier Ausgaben in ihrer Reihenfolge. Erklären Sie, welche Werte beim Aufruf kopiert werden und warum die Variablen in `main` ihre Werte behalten.

b) Setzen Sie in IntelliJ einen Haltepunkt am Aufruf und gehen Sie mit **Step Into** in die Methode. Skizzieren Sie die Variablen beider Aufrufe und die Referenzpfeile zu den Texten: direkt nach dem Eintritt in `aendere`, nach den beiden Zuweisungen und nach der Rückkehr.

c) Schreiben Sie stattdessen `String erweitere(String text)`, das den um ein Ausrufezeichen ergänzten Text zurückgibt. Zeigen Sie, wie `main` das Ergebnis übernehmen muss, damit seine Variable `text` anschließend diesen Text referenziert.

06 Eine Suche nicht zu früh beenden | Combined

Diese Methode soll prüfen, ob ein Zeichen im Text vorkommt. Sie enthält einen logischen Fehler.

```
boolean enthaelt(String text, char gesucht) {
    for (int i = 0; i < text.length(); i++) {
        if (text.charAt(i) == gesucht) {
            return true;
        } else {
            return false;
        }
    }
    return false;
}
```

a) Vergleichen Sie erwartetes und tatsächliches Ergebnis für `enthaelt("Methoden", 'h')`, `enthaelt("Methoden", 'M')`, `enthaelt("Methoden", 'x')` und `enthaelt("", 'h')`. Verfolgen Sie den fehlerhaften Fall im Debugger.

b) Korrigieren Sie die Methode: Ein Treffer beendet die Suche sofort. Erst nach dem Prüfen aller Zeichen darf sie `false` zurückgeben. Erklären Sie auch, weshalb das `return` nach der Schleife nötig ist.

Combined: Unabhängige Berechnungen und der ggT

07 Zwischenwerte lokal halten | Combined

Die Variable `summe` steht hier außerhalb aller Methoden. In `main` werden die beiden gezeigten Aufrufe nacheinander ausgeführt.

```
int summe = 0;

int addiereBis(int grenze) {
    for (int i = 1; i <= grenze; i++) {
        summe += i;
    }
    return summe;
}

IO.println(addiereBis(3));
IO.println(addiereBis(3));
```

a) Welche Werte werden ausgegeben? Untersuchen Sie im Debugger, welchen Wert `summe` zu Beginn jedes Aufrufs hat. Weshalb beeinflusst der erste Aufruf das Ergebnis des zweiten? Ersetzen Sie anschließend `int summe = 0;` durch `int summe;` und starten Sie das Programm neu. Ändern sich die Ausgaben? Begründen Sie.

b) Überarbeiten Sie die Methode so, dass sie ausschließlich lokale Variablen und ihren Parameter verwendet. Jeder Aufruf soll unabhängig von früheren Aufrufen die Summe von 1 bis `grenze` berechnen. Für `grenze = 0` ist das Ergebnis 0; erlaubt sind Werte von 0 bis 1000. Entfernen Sie danach auch bei der lokalen Summe die Zuweisung des Startwerts. Lässt sich das Programm noch kompilieren? Begründen Sie den Unterschied zu a) und stellen Sie die Zuweisung wieder her.

c) Testen Sie die Grenzen 0, 1, 3 und 1000 sowie wiederholte Aufrufe mit demselben Wert. Nennen Sie ein Beispiel, in dem ein Zustand bewusst über mehrere Aufrufe hinweg erhalten bleiben soll.

08 Den ggT mit Modulo berechnen | Combined

Der bekannte ggT-Algorithmus zieht wiederholt die kleinere Zahl von der größeren ab. Eine schnellere Variante ersetzt das Zahlenpaar `(a, b)` durch `(b, a % b)`. Sobald `b` gleich 0 ist, enthält `a` den ggT. Die ursprünglich übergebenen Zahlen sind positiv und höchstens 1000.

a) Implementieren Sie `int ggtSubtraktion(int a, int b)` mit einer Schleife nach dem bekannten Verfahren. Schreiben Sie anschließend `int ggtModulo(int a, int b)` für das Verfahren mit Modulo, ebenfalls mit einer Schleife.

b) Schreiben Sie `int ggtRekursiv(int a, int b)` als rekursive Modulo-Variante ohne Schleife. Welche Bedingung beendet die Rekursion? Warum ist sie endrekursiv? Beachten Sie, dass im Verlauf der Berechnung 0 auftreten kann.

c) Testen Sie alle drei Methoden mit `(84, 30)`, `(17, 13)`, `(42, 42)` und `(100, 1)`. Vergleichen Sie beim letzten Paar die Anzahl der Schleifendurchläufe beider iterativen Varianten. Erklären Sie, warum der Rest bei der Division viele einzelne Subtraktionen ersetzt und der ggT erhalten bleibt.

d) Lesen Sie in `main` zwei Zahlen ein und geben Sie die drei Ergebnisse aus. Die ggT-Methoden selbst sollen weder Eingaben lesen noch etwas ausgeben.

Challenge: Rekursion und Endrekursion

09 Ziffern rekursiv zählen | Challenge

Schreiben Sie `int zaehleZiffer(int ziffer, int zahl)`. Die Methode zählt, wie oft `ziffer` in der dezimalen Darstellung von `zahl` vorkommt. Es gilt: `ziffer` liegt zwischen 0 und 9, `zahl` zwischen 0 und `Integer.MAX_VALUE`. Zusätzliche führende Nullen werden nicht mitgezählt. Die Zahl 0 besteht aus genau einer Ziffer 0.

a) Implementieren Sie die Methode rekursiv, ohne Schleifen und ohne Umwandlung in einen `String`. Mit `% 10` erhalten Sie die letzte Ziffer; mit ganzzahligem `/ 10` entfernen Sie diese. Legen Sie zuerst den Abbruchfall fest. Bei dieser ersten Variante soll die Zählung nach der Rückkehr aus dem rekursiven Aufruf fortgesetzt werden.

Prüfen Sie Ihre Methode mit diesen Aufrufen:

Aufruf	Erwarteter Rückgabewert
`zaehleZiffer(7, 70701277)`	4
`zaehleZiffer(0, 70701277)`	2
`zaehleZiffer(7, 111)`	0
`zaehleZiffer(0, 0)`	1
`zaehleZiffer(7, 0)`	0
`zaehleZiffer(0, 1000)`	3

b) Zeigen Sie für `zaehleZiffer(7, 707)` den Aufbau und Abbau des Laufzeitstacks, einschließlich des aufrufenden `main`. Halten Sie wie in der Vorlesung pro Schritt die aktive Codezeile, Parameterwerte, die Stelle der späteren Fortsetzung und die noch ausstehende Rechnung fest. Welche Werte werden jeweils zurückgegeben?

c) Erstellen Sie in einer zweiten Datei `K05Aufgabe09Endrekursiv.java` eine endrekursive Variante. Der Aufruf mit zwei Argumenten soll erhalten bleiben. Verwenden Sie dazu eine überladene Hilfsmethode mit einem dritten Parameter für die bisher gezählten Treffer. Achten Sie erneut auf die Zahl 0.

d) Verfolgen Sie auch diese Variante für `(7, 707)`. Was wird beim Aufbau des Stacks berechnet? Was geschieht beim Abbau noch mit dem Ergebnis? Vergleichen Sie beide Abläufe und begründen Sie, warum die Rekursion endet. Spart Endrekursion in Java garantiert Stackplatz?

Challenge: Ein größeres Problem in Methoden zerlegen

10 Eine deutsche IBAN berechnen | Challenge

Eine IBAN ist eine internationale Kontokennung. Eine deutsche IBAN hat 22 Zeichen: `DE`, eine zweistellige Prüfzahl, acht Ziffern Bankleitzahl (BLZ) und zehn Ziffern Kontonummer. Kürzere Kontonummern werden für die Standardregel links mit Nullen ergänzt.

Praxisbezug: Wir behandeln die deutsche Standardregel. Einzelne Banken verwenden bei der Umrechnung besondere Regeln, die hier entfallen. Eine passende Prüfzahl bestätigt nicht, dass ein Konto existiert. Die folgenden Daten sind reine Rechenbeispiele.

Hintergrund: Deutsche Bundesbank – IBAN-Regeln

Vorgaben: Das Land ist fest `DE`. Die BLZ besteht aus genau acht Ziffern, die Kontonummer aus einer bis zehn Ziffern. Beide werden als `String` ohne Leerzeichen verarbeitet, damit führende Nullen erhalten bleiben.

Verfahren zur Berechnung der Prüfzahl

1. Ergänzen Sie die Kontonummer links auf zehn Ziffern. Hängen Sie diese an die BLZ. Die entstandene nationale Kontokennung heißt BBAN und hat 18 Ziffern.

2. Für die Prüfrechnung werden Buchstaben in Zahlen umgewandelt: A entspricht 10, B entspricht 11, …, Z entspricht 35. Für `DE` ergibt sich daher `1314`. Hängen Sie an die BBAN zunächst `1314` und dann `00` als Platzhalter für die Prüfzahl an.

3. Bestimmen Sie den Rest dieser 24-stelligen Dezimalzahl bei Division durch 97. Ziehen Sie den Rest von 98 ab. Schreiben Sie das Ergebnis immer zweistellig, nötigenfalls mit einer führenden Null.

4. Setzen Sie die IBAN aus `DE`, dieser Prüfzahl und der BBAN zusammen.

Beispiel: Für BLZ `12345678` und Kontonummer `123456` entsteht die BBAN `123456780000123456`. Die Prüfrechnung verwendet `123456780000123456131400`. Der Rest ist 58, die Prüfzahl also 40. Das Ergebnis lautet `DE40123456780000123456`.

a) Zerlegen Sie das Verfahren in sinnvolle Teilaufgaben. Notieren Sie für jede Hilfsmethode Namen, Parameter und Rückgabetyp. Die gesamte Berechnung soll über `String erzeugeIban(String blz, String konto)` aufrufbar sein. Einlesen und Ausgeben gehören ausschließlich nach `main`.

b) Für Schritt 3 brauchen wir nur den Divisionsrest, nicht die ganze 24-stellige Zahl. Diese passt nicht in `long`. Schreiben Sie deshalb `int rest97(String ziffern)`: Die Methode erhält die Zahl als Text und gibt ihren Rest bei Division durch 97 zurück.

Beginnen Sie mit `rest = 0` und lesen Sie die Ziffern von links nach rechts. Speichern Sie immer nur den bisherigen Rest. Für jedes Zeichen an Position `i` führen Sie diese beiden Schritte aus:

```
int z = ziffern.charAt(i) - '0';
rest = (rest * 10 + z) % 97;
```

Die erste Zeile wandelt das Zeichen in seinen Zahlenwert um, z. B. `'7'` in 7. `rest * 10 + z` hängt die nächste Ziffer an den bisherigen Rest an. `% 97` liefert den neuen Rest. Nach der letzten Ziffer geben Sie `rest` zurück.

Rechnen Sie zunächst das kurze Beispiel `"1234"` von Hand durch: Notieren Sie nach jeder Ziffer den Rest und vergleichen Sie das Endergebnis mit `1234 % 97`. Testen Sie anschließend Ihre Methode damit und mit der 24-stelligen Ziffernfolge aus dem Beispiel oben; dort muss der Rest 58 entstehen.

Wie groß können der Rest nach `% 97` und der Zwischenwert `rest * 10 + z` davor höchstens werden? Begründen Sie damit, warum `int` für diese Rechnung ausreicht.

Challenge: IBAN berechnen und die Prüfzahl kontrollieren

Fortsetzung von Aufgabe 10

c) Implementieren Sie die übrigen Hilfsmethoden und `erzeugeIban`. Nutzen Sie zum Ergänzen führender Nullen eine wiederverwendbare Methode, die einen Text und die gewünschte Länge erhält. Verwenden Sie sie für Kontonummer und Prüfzahl. Schreiben Sie für `erzeugeIban` und `rest97` Doc-Comments mit `@param` und `@return`.

d) Lesen Sie in `main` BLZ und Kontonummer mit `IO.readln()` ein und geben Sie die berechnete IBAN aus. Testen Sie die folgenden Fälle. Warum sind der zweite und der dritte Fall zusätzlich zum ersten sinnvoll?

BLZ	Kontonummer	Erwartete IBAN
`12345678`	`123456`	`DE40123456780000123456`
`12345678`	`1234567890`	`DE87123456781234567890`
`12345678`	`5`	`DE06123456780000000005`

Ein beispielhafter Programmlauf

```
BLZ: 12345678
Kontonummer: 123456
IBAN: DE40123456780000123456
```

e) Ergänzen Sie `boolean hatGueltigePruefzahl(String iban)`. Die Eingabe besteht hier immer aus `DE` und 20 Ziffern ohne Leerzeichen. Für die Kontrolle werden die ersten vier Zeichen ans Ende verschoben und D bzw. E durch 13 bzw. 14 ersetzt. Die Prüfzahl bleibt dabei erhalten. Die entstandene Ziffernfolge muss bei Division durch 97 den Rest 1 ergeben. Verwenden Sie Ihre Methode `rest97` erneut.

Beispiel zur Kontrolle der Prüfzahl

Ausgangswert: `DE40123456780000123456`.

1. Verschieben Sie die ersten vier Zeichen `DE40` ans Ende:

```
12345678000123456DE40
```

2. Ersetzen Sie `D` durch `13` und `E` durch `14`:

```
123456780000123456131440
```

3. Übergeben Sie diese Ziffernfolge als Text an `rest97`. Der zurückgegebene Rest ist 1. Deshalb liefert `hatGueltigePruefzahl` hier `true`.

Die Prüfzahl `40` bleibt erhalten. Anders als beim Erzeugen der IBAN wird sie für diese Kontrolle nicht durch `00` ersetzt.

Testen Sie alle drei berechneten IBANs und anschließend `DE41123456780000123456`. Welche Änderung enthält der letzte Fall? Erklären Sie, was Ihre Prüfung über diese Zeichenfolge aussagt und was sie über ein tatsächliches Bankkonto nicht feststellen kann.

Hinweise zu den String-Operationen: `substring(4)` liefert den Text ab Index 4 bis zum Ende; `substring(0, 4)` liefert die ersten vier Zeichen. Einen Zahlenwert können Sie mit `"" + wert` in einen Text umwandeln. Die gesamte lange Ziffernfolge wird zu keinem Zeitpunkt in einen Ganzzahltyp eingelesen.